

While you can find PostgreSQL extensions all around the internet, chances are you will interface with PGXN almost every time you need a new extension. PGXN is not a simple website or a code repository, but it is a detailed platform made of four parts: a search engine, an extension manager, an **application programming interface (API)**, and a client.

The **search engine** allows users to search the PGXN content for a specific extension. The **manager** is responsible for accepting new extensions (or new extension versions) and letting users obtain them (that is, distributing the extensions). The **API** defines how applications can interact with the manager and the search engine, and therefore how a client can be built. There are two main clients available—the PGXN website and the `pgxnclient` command-line application. The `pgxnclient` application is probably the most efficient way to get and install an extension, which will be detailed in the following subsections; however, you can interact directly with the PGXN website to search for and download extensions too. You will see an example of using the PGXN website later in the chapter.

Extensions are built on top of the **PostgreSQL eXtension System (PGXS)**, which is a basic set of rules an extension must adhere to in order to expose a common manageable interface. In particular, PostgreSQL provides a uniform Makefile that every extension should use to provide a set of common functionalities to install, upgrade, and remove an extension. You can inspect the PGXS base Makefile, finding its location with `pg_config`:

```
$ pg_config --pgxs  
/usr/lib/postgresql/16/lib/pgxs/src/makefiles/pgxs.mk
```

`pgxs.mk` is the base Makefile that provides common functionalities to every extension, and its usage will become more clear when we show you how to create an extension from scratch.

Extension components

An extension is made up of two main components—a **control file** and a **script file**:

- The **control file** provides information about the extension and how to manage it, for instance, where and how to install it, how to upgrade it, and so on. The control file is somehow the metadata of the extension.
- The **script file** is a SQL file that contains statements to create database objects that are part of the extension. To some extent, this is the content of the extension. The script file can, in turn, load other files that complete the extension, like a shared library and the like.

When you ask PostgreSQL to install an extension, the system inspects the control file to get information about the extension, ensures the extension has not already been installed, and then proceeds to execute the script file within a transaction. As a result, you have the extension available in your database.

Every extension has a version so that you can decide precisely which version to install. If you do not specify a version, PostgreSQL will assume you want the latest version available.

Extensions are installed in the share directory of the cluster, usually found by executing the `pg_config` command with the `--sharedir` option. Here's an example:

```
$ pg_config --sharedir
/usr/share/postgresql/16
```

All the files that make up the extension will be placed in the shared directory, and the cluster expects the files to be available there to the user that runs the cluster (usually the operating system user `postgres`). Once the files are available to the cluster, the extension must be selectively installed in every database that needs it; remember that PostgreSQL provides very strong isolation between databases, and therefore an extension loaded into a database is not automatically available in another database. However, please remember that template databases (see *Chapter 2*) can be used as a skeleton for newly created databases, and therefore once you install an extension in a template database, you will find this extension already available in all the other created databases.

The control file

An extension control file must have a name that is related to the extension and the `.control` suffix. For example, a valid name could be `learnpg.control`.

The control file is a text file where you can specify directives, which are instructions and metadata to let PostgreSQL handle the extension installation. Every directive has a name and a value. The most common directives are as follows:

- `directory` specifies the path to the extension script path.
- `default_version` specifies the version of the extension to install when the user does not specify any.
- `comment` is a description of the extension and its aim.
- `requires` is an optional list of other extensions needed to install and use this, and therefore represents a dependency list.

- `schema` is a SQL schema into which extension objects will be installed.
- `relocatable` indicates whether the extension can be moved into a user-selected schema.
- `superuser` indicates whether the extension can also be installed by non-superuser accounts (defaults to yes, meaning that only superusers can install the extension).

There must be at least one control file per extension, and such a file is known as the *main control file*. However, an extension can have additional control files (named *secondary control files*).

Every secondary control file must target a specific version and must have the same name as the main control file with the version number prefixed with double dashes; for instance, if the main control file is `learnpg.control`, the secondary files could be `learnpg--1.1.control`, `learnpg--1.2.control`, and so on.

The script file

The script file contains plain SQL used to create extension objects. An extension object could be a table, a trigger, a function, or a binding for an external language.

Every script file must be named after the extension name and with a suffix of `.sql`; the version of the extension is specified with a number preceded by a double dash. As an example, the `learnpg--1.0.sql` file creates objects for version 1.0 of the extension.

There must be at least one script file per extension, but it is possible to specify more than one. In such cases, every additional file must include the version to upgrade from and the final target version. For example, the `learnpg--1.0-1.1.sql` file provides an upgrade from version 1.0 to version 1.1.



Every extension has a version number. Higher version numbers mean an upgrade of the extension, while lower numbers mean a downgrade of the extension. For example, version 1.2 is an upgrade of 1.1, while 1.0 is a downgrade.

As already specified, the script file is executed in a transaction and therefore cannot interact with the transaction boundaries (that is, it can issue neither a `COMMIT` nor a `ROLLBACK`). Similarly, when executing in a transaction, a script file is prevented from executing anything that cannot be executed in a transaction block (for example, utility commands such as `VACUUM`).

Managing extensions

Every extension is managed at a database level, meaning that every database that needs an extension must manage such an extension life cycle. In other words, there is no per-cluster way of managing an extension and applying it to every database within the cluster.

Extensions are mainly managed by three SQL statements: `CREATE EXTENSION`, `DROP EXTENSION`, and `ALTER EXTENSION`, to respectively install an extension in a database, remove the extension from the database, and modify extension attributes or upgrade them.

Every extension is specified by a mnemonic and a version; if a version is not specified, PostgreSQL assumes you want to deal with the latest available version or the one that is already installed.

In the following subsections, each of the three management statements will be explained.

Creating an extension

The `CREATE EXTENSION` statement allows you to install an existing extension in the current database.

The synopsis of the statement is as follows:

```
CREATE EXTENSION [ IF NOT EXISTS ] extension_name
    [ WITH ] [ SCHEMA schema_name ]
    [ VERSION version ]
    [ CASCADE ]
```

The extension name is the mnemonic for the extension, and as you can see, you can specify the version number of the extension to install. If the extension depends on any other extension, the `CASCADE` option allows the system to automatically execute a recursive `CREATE EXTENSION` for the dependency. You can decide which schema the extension objects must be placed into, and of course, that makes sense only for such extensions that can be relocated.

As you can imagine, `IF NOT EXISTS` allows the command to gracefully fail if the extension has been already installed. More precisely, it does nothing if the extension has already been installed in the database.

In order to better see how `CREATE EXTENSION` works, assume we want to install the PL/Perl procedural language in the `forumdb` database; since the PL/Perl extension is available as the PostgreSQL `contrib` module, you should have the extension already available within the cluster.

Therefore, in order to install it, you have to do the following:

```
forumdb=# CREATE EXTENSION plperl;
CREATE EXTENSION
```

Please note that the PL/Perl extension (mnemonic `plperl`) requires installation using the database administrator. If you try to install the same extension again, the command fails unless you use the `IF NOT EXISTS` clause:

```
forumdb=# CREATE EXTENSION plperl;
ERROR:  extension "plperl" already exists

forumdb=# CREATE EXTENSION IF NOT EXISTS plperl;
NOTICE:  extension "plperl" already exists, skipping
CREATE EXTENSION
```

As another easy example, we can install a specific version of the `pg_stat_statements` extension:

```
forumdb=# CREATE EXTENSION pg_stat_statements VERSION '1.10';
CREATE EXTENSION
```



Please note that `pg_stat_statements` requires a change to the `shared_preload_libraries` configuration parameter, which in turn requires the cluster to be restarted. In the Docker image of this chapter, the `shared_preload_libraries` setting is already configured appropriately for you to use the `pg_stat_statements` extension. If you want to manually change the PostgreSQL configuration in order to use `pg_stat_statements`, you need to edit the `postgresql.conf` configuration file or execute:

```
ALTER SYSTEM SET shared_preload_libraries TO 'pg_stat_statements';
```

Then restart the cluster.

Viewing installed extensions

In the `psql` terminal, it is possible to get a list of installed extensions with the `\dx` special command:

```
forumdb=# \dx

                                List of installed extensions
  Name          | Version | Schema |
Description
```

```

-----+-----+-----+-----
pg_stat_statements | 1.10      | public      | track execution statistics
of all SQL statements executed
plperl             | 1.0       | pg_catalog  | PL/Perl procedural language
plpgsql            | 1.0       | pg_catalog  | PL/pgSQL procedural language
(3 rows)

```

The very same information can be obtained from the special `pg_extension` catalog, which can be joined with `pg_namespace` to extract human-readable information about the schema the extension is living in:

```

forumdb=# SELECT x.extname, x.extversion, n.nspname
          FROM pg_extension x JOIN pg_namespace n
          ON n.oid = x.extnamespace;

 extname      | extversion | nspname
-----+-----+-----
plpgsql       | 1.0        | pg_catalog
plperl        | 1.0        | pg_catalog
pg_stat_statements | 1.10      | public
(3 rows)

```

Finding out available extension versions

It is possible to inspect the cluster to get information about available extension versions, which means versions you can actually install in a database. The special `pg_available_extension_versions` catalog allows you to get all the available versions for any available extension. As an example, the `pg_stat_statements` extension has the following values available in the cluster:

```

forumdb=# SELECT name, version
          FROM pg_available_extension_versions
          WHERE name = 'pg_stat_statements';

 name      | version
-----+-----
 name      | version
-----+-----
pg_stat_statements | 1.4

```

```
pg_stat_statements | 1.5
pg_stat_statements | 1.6
pg_stat_statements | 1.8
pg_stat_statements | 1.9
pg_stat_statements | 1.7
pg_stat_statements | 1.10
```

It is useful to know that the `pg_stat_statements` extension can be installed in a version between 1.4 and 1.10 depending on your needs.



You should always install the latest version of an extension, that is, the one with the highest version number, unless you are forced to install a specific version for backward compatibility.

Altering an existing extension

The `ALTER EXTENSION` statement is very rich and complex and allows you to fully modify an existing extension. The statement allows four main changes to an existing extension:

- Upgrading the extension to a new version
- Setting the schema of a relocatable extension
- Adding a database object to the extension
- Removing a database object from the extension

In order to upgrade an already installed extension, you must specify the `UPDATE` clause, specifying the target version number. As an example, consider the `pg_stat_statements` extension presented before, and assume we install version 1.6 of it for the sake of upgrading it. In order to update the extension to version 1.10, it is possible to issue an `ALTER EXTENSION` statement as the following example shows:

```
forumdb=# CREATE EXTENSION
          pg_stat_statements WITH VERSION '1.6';
CREATE EXTENSION

forumdb=# ALTER EXTENSION pg_stat_statements
          UPDATE TO '1.10';
ALTER EXTENSION
```

```

forumdb=# \dx pg_stat_statements

                                List of installed extensions
   Name          | Version | Schema |
Description
-----+-----+-----+-----
 pg_stat_statements | 1.10   | public | track execution statistics of
all SQL statements executed
(1 row)

```

Moving a relocatable extension from one schema to another is done by specifying the SET SCHEMA clause, for example:

```

forumdb=# ALTER EXTENSION pg_stat_statements SET SCHEMA my_schema;
ALTER EXTENSION

forumdb=# \dx pg_stat_statements

                                List of installed extensions
   Name          | Version | Schema |
Description
-----+-----+-----+-----
 pg_stat_statements | 1.10   | my_schema | track execution statistics of
all SQL statements executed
(1 row)

```

That will move all the extension objects into the my_schema schema, which has to exist before the extension is relocated.

If you want to remove an existing database object from one extension, for instance, a table, you can use the DROP clause followed by the type of the object and, of course, its name. It is important to understand that removing an object from an extension will not remove the object from the database; rather, it will unlink the object lifecycle from the lifecycle of the extension. In other words, an object that drops out from an extension becomes a normal database object. As an example, if we remove the pg_stat_statements view from the extension with the same name, we can specify the object type (VIEW) after the DROP clause, as follows:

```

forumdb=# ALTER EXTENSION pg_stat_statements
          DROP VIEW pg_stat_statements;
ALTER EXTENSION

```


If the extension has been relocated, it's necessary to specify the qualified name for every object, therefore prefixing the object name with the current extension schema. For example, after having relocated the extension into the `my_schema` namespace, the above command becomes:

```
forumdb=# ALTER EXTENSION pg_stat_statements
          DROP VIEW my_schema.pg_stat_statements;
ALTER EXTENSION
```

The result of the above `DROP VIEW` command is that the view initially created at extension deployment time is still there in the database and can be queried regularly; but this view is now an object with an independent lifecycle from the extension itself.

It is easy to see how the view is still available by simply querying it:

```
forumdb=# SELECT count(*) FROM pg_stat_statements;
count
-----
      2
(1 row)
```

Of course, it is possible to add a new object to an extension with the `ADD` clause, which works as the opposite of the `DROP` one and requires the type and name of the object. For instance, to add the `pg_stat_statements` view back to the extension, it is possible to do the following:

```
forumdb=# ALTER EXTENSION pg_stat_statements
          ADD VIEW pg_stat_statements;
ALTER EXTENSION
```

You can also add your own objects to the extension. So, for example, adding a new table to the extension means that the extension will undergo the extension life cycle:

```
forumdb=# CREATE TABLE t_ext( i int, t text );
forumdb=# ALTER EXTENSION pg_stat_statements
          ADD TABLE t_ext;
ALTER EXTENSION
```

The `t_ext` table is now part of the extension, and as such, it cannot be manipulated anymore with statements that do not take the extension into account.

For instance, if you try to delete the table, PostgreSQL will prevent you from damaging the extension:

```
forumdb=# DROP TABLE t_ext;  
ERROR:  cannot drop table t_ext because extension pg_stat_statements  
requires it  
HINT:  You can drop extension pg_stat_statements instead.
```

This last example properly showcases the power of extensions: all objects belonging to an extension are managed as a whole, and therefore cannot be accidentally managed or removed because they are dependent on each other. This means that, in order to delete the above table, you either need to remove it from the extension before dropping the table itself or delete the whole extension.

Removing an existing extension

`DROP EXTENSION` deletes an extension from the current database. The synopsis of the statement is the following:

```
DROP EXTENSION [ IF EXISTS ] name [, ...] [ CASCADE | RESTRICT ]
```

The command supports the `IF EXISTS` clause, as many other statements do. Moreover, it is possible to specify more than one extension to be removed from the database.

The `CASCADE` option also removes database objects that depend on the objects of the extension, while its counterpart, `RESTRICT`, makes the command fail if there are other objects that still depend on this extension. Moreover, it is possible to drop more than one extension at the same time.

As an example, the following statement removes two extensions in a single pass, also removing all the objects that depend on those extensions:

```
forumdb=# DROP EXTENSION plperl, plpgsql CASCADE;  
NOTICE: drop cascades to function get_max(integer,integer)  
DROP EXTENSION
```

As you can see, since the user-defined `get_max()` function was dependent on one of the two extensions, the `CASCADE` option made the process drop the function too.